The Dominium Project Vision Book

Printable Desktop Edition

Status: DERIVED

Promotion Status: not_promoted

2026-06-03

Contents

Preface: What This Book Is and Is Not

This book is a derived project-vision synthesis. It gathers the committed project vision corpus, the external scratchpad corpus, and current repo authority into one human-readable account of what Dominium is trying to become. It is meant to be read by a person who wants the shape of the project without reading every old chat, report, manifest, and handoff package.

The book is not canon. It does not change contracts, schemas, implementation, release state, or queue state. Its job is to explain the project, preserve useful historical intent, show the difference between current reality and ambition, and make future decisions easier.

The reader should keep one rule in mind throughout: current repo authority wins over archive memory. The archive can explain why an idea matters. It cannot silently make that idea current truth.

The comparison step matters because each corpus has a different bias. The committed repo corpus is safer and more authority-aware. The scratchpad corpus is broader in uploaded-package coverage and often clearer about product ambition. A useful book should not choose one blindly. It should let the repo corpus control current status while letting the scratchpad improve the language of the long-horizon vision.

A reader should use this book as an orientation and decision aid. It is meant to reduce the cost of understanding the project, not to remove the need for review. When the book says a direction is settled, that means settled as planning synthesis unless a current repo authority source also says it is binding. When the book says a direction is attractive, that means it has recurring support in the archive, not that implementation is open.

1. The Project in One Sentence

Dominium is a deterministic, authority-governed simulation product ecosystem built on Domino, a portable substrate that protects truth, replay, provenance, capability refusal, and long-term compatibility while allowing products, tools, renderers, providers, packs, and worlds to evolve around it.

That sentence is long because the project is layered. It is not only a game, not only an engine, not only a toolchain, and not only a documentation system. It is a project that wants those surfaces to coexist without confusing which layer owns authoritative truth.

The simplest mental model is this: Domino keeps the lawful substrate stable; Dominium turns that substrate into a product universe; Workbench, AIDE, tools, launchers, renderers, packs, and future worlds inspect or project the substrate without replacing it.

This framing also explains why many previous outputs felt unsatisfying. A manifest can prove coverage, but it cannot explain the project. A source appendix can preserve documents, but it cannot decide what they mean. A register can classify evidence, but it cannot give a reader the project in one sitting. This book is meant to do the missing work: turn the corpus into a usable mental model.

2. Why Dominium Exists

The archive repeatedly returns to the same frustration: large simulation projects collapse when presentation, convenience, provider choices, and ad hoc tooling start to own truth. Dominium exists to avoid that collapse. It wants a rich product and a large simulation horizon, but it wants those ambitions built on a narrow, inspectable authority model.

The project is attracted to scale: planets, history, logistics, civilization, institutions, automation, factories, observability, editor surfaces, native interfaces, and long-lived content. Those ambitions require a foundation that can survive renderer swaps, platform changes, old saves, mod packs, provider differences, and future tooling.

That is why the project repeatedly prefers determinism, explicit refusal, compatibility metadata, provenance, and process-only mutation. These ideas are not bureaucracy. They are the protection that makes large ambition feasible.

The purpose is not only technical neatness. It is continuity. If Dominium is meant to outlive early providers, early renderers, early shells, and early tooling experiments, then its core meaning has to be portable. The project exists to make a world and product suite that can change presentation without changing law.

3. The Core Distinction: Domino and Dominium

The strongest reconciliation across both corpora is the Domino/Dominium split. Domino is the reusable deterministic substrate. Dominium is the official product-facing universe that uses the substrate to become a game, simulation, tool suite, and long-horizon world.

Without that distinction, the archive appears contradictory. Some conversations talk like a game design session. Others talk like engine architecture. Others talk like repository governance, product shell design, Workbench planning, or world simulation theory. The distinction explains the apparent conflict: the project has a substrate layer and a product layer, and both matter.

Domino should carry the portable law: deterministic execution, stable contracts, provider boundaries, refusal, serialization, validation, and replay. Dominium should carry authored meaning: official products, domains, worlds, player experience, content packs, launcher/setup flows, and user-facing tools.

This does not mean the boundary is fully specified. The exact API, package layout, provider ABI, first playable slice, and promotion path remain open. But the conceptual split is settled enough to plan around.

A future spec should preserve this distinction even if the names evolve. The important rule is dependency direction. Lower substrate law should not depend on the official product's current UI, current game loop, or current content ambitions. The product can specialize the substrate; it should not trap it.

4. The Stable Center: Deterministic Truth

The stable center of the project is deterministic truth. In current repo language, authoritative truth changes through lawful process execution. Rendering, UI, perception, tooling, and convenience surfaces do not mutate truth directly.

The archive adds practical reasons for this rule. A large simulation needs replay, old save compatibility, reproducible validation, portable providers, and long-lived content. If a renderer, platform backend, UI action, or scripting shortcut becomes the place where truth is corrected or improvised, the whole project loses the ability to prove what happened.

Determinism is therefore a product feature, an engineering rule, and a governance principle at the same time. It affects numeric choices, timekeeping, serialization, testing, content packs, multiplayer feasibility, provider law, and future automation.

The long-horizon vision can include rich presentation and broad world simulation only if this center remains narrow.

This has a design consequence for every future feature. A feature is not complete merely because it appears on screen. It is complete only when its authoritative effects can be explained, replayed, validated, and refused when unsupported. That standard is higher than ordinary game prototyping, but it is what lets the project remain inspectable.

5. Authority, Law, Capability, and Refusal

Authority is the project's way of refusing ambiguity. The current repo gives precedence to canon, glossary, governance instructions, contracts, schemas, queue state, and validated artifacts. The corpus and old chats sit below that order. They can advise, but they cannot overrule.

Capability and refusal are the runtime form of the same idea. A provider, pack, tool, or surface should declare what it can do. If it cannot do something, it should refuse or degrade explicitly. Silent fallback is dangerous because it hides broken assumptions until determinism, replay, or compatibility fails.

The archive strongly supports this posture. It repeatedly favors provider declarations, explicit compatibility metadata, validation, and visible refusal over magical convenience. That is one of the clearest places where long-horizon ambition and current repo doctrine reinforce each other.

Refusal is especially important because Dominium wants rich optionality. Optional packs, providers, platforms, renderers, and tools create many ways for something to be absent. A disciplined system says what is absent and why. A weak system pretends the absence did not matter.

6. Simulation, Runtime, Engine, and Product Boundaries

Dominium needs a runtime, an engine-like substrate, products, tools, and eventually user-facing experiences. The risk is letting those labels blur. A product shell should not become the simulation law. A renderer should not become the state model. A tool should not get private mutation authority because it is convenient.

The current repo blocks broad implementation in several areas, including broad Workbench UI, renderer implementation, native GUI, provider runtime, package runtime, gameplay, and release publication. That does not erase the archive ambition. It means the ambition must be sequenced through review and explicit queue opening.

The final design should allow a headless path, product shells, launch/setup flows, client/server surfaces, and tool surfaces to share the same lawful substrate. The key is not which product comes first. The key is preserving dependency direction: products consume and operate the substrate; they do not redefine it.

The word engine should therefore be used carefully. Domino may behave like an engine or framework, but the project should avoid importing the habits of engine-owned truth. Likewise, Dominium may become a game, but its game surface should not erase the substrate distinction that makes portability and replay possible.

7. Presentation: Renderer, UI, Native GUI, and Projections

Presentation is one of the richest and most dangerous areas in the archive. Conversations explore renderers, UI layers, native GUI, provider APIs, external engines, and product surfaces. The useful synthesis is simple: presentation should be replaceable projection.

A renderer can be powerful. A native GUI can make tools usable. A commercial engine can accelerate visualization. But none of them should own authoritative simulation truth. The project can use outside libraries aggressively if they remain providers, clients, adapters, tools, or references.

This boundary has a practical consequence. Renderer work cannot be treated as a shortcut into simulation design. If presentation stores truth or patches truth, replay and validation are weakened. The right sequence is to define contracts and provider refusal before letting presentation become a product dependency.

The external scratchpad usefully sharpened this point by describing outside systems as providers, clients, adapters, tools, or references. That vocabulary keeps enthusiasm practical. The project can learn from Unreal, raylib, SDL, native GUI frameworks, and other systems without letting any of them become the project's law.

8. Workbench, AIDE, and the Operator Surface

Workbench and AIDE are not just optional tools in the archive. They are the proposed operator surface for a governed project: a way to inspect state, manage tasks, preserve evidence, route Codex work, review generated outputs, and keep changes tied to authority.

The current repo treats many Workbench ambitions as blocked unless a scoped task opens them. That caution is appropriate. A Workbench that bypasses governance would be worse than no Workbench. The value of Workbench is that it can make lawful operation easier, not that it can sidestep review.

The mature version of this layer should help humans and agents see what is current, what is historical, what is blocked, what needs validation, and what can be safely done next. That makes it a product surface and a governance surface at once.

A good operator surface should make uncertainty visible. It should show what is current, what is advisory, what is stale, what is blocked, what needs validation, and what has a source trail. In that sense Workbench is not merely an editor; it is a way to make the project's governance usable.

9. Packs, Providers, Content, and Modding

The project wants extension without hidden reinterpretation. Packs, providers, content, modding, and registries should make the system more capable while preserving law. Missing packs should produce explicit refusal or documented degradation, not silent fallback.

The archive repeatedly connects this idea to compatibility. Old saves, old packs, mods, authored content, manifests, and user data should not break because a provider changed or a convenience migration quietly reinterpreted meaning.

This creates a demanding but useful standard: content should be data-driven and extensible, but contracts must say what that data means. Providers should be replaceable, but replacement should not change authoritative behavior without being visible and validated.

This principle also helps with modding. A modding ecosystem becomes dangerous when every extension can reinterpret core behavior silently. It becomes powerful when extensions can declare requirements, depend on capabilities, refuse missing support, and remain compatible with the substrate's contract language.

10. World Model, Time, Space, and Scale

The long-horizon world model is broad. It includes timekeeping, scale, space, celestial systems, domains, world generation, persistence, and the difference between what truly exists and what a player or system can perceive.

Current repo authority already supports truth, perception, and rendering as separate layers. The archive expands that separation into a vision of worlds that can be large, sparse, inspectable, and gradually refined. Not every possible object must be simulated at full fidelity at all times. But whatever is authoritative must be lawful and replayable.

Time and scale are not just flavor. They shape save compatibility, world history, replay, scheduling, logistics, and old-instance resilience. This is why 2038-style time concerns and deterministic chronology keep appearing in the archive.

The world model should also avoid premature density. A large deterministic universe does not require full simulation of everything everywhere. It requires clear rules for what exists, what is refined, what is perceived, what is rendered, and what can be reconstructed or advanced lawfully.

11. World Generation and Long-Horizon Simulation Ambition

World generation is where the archive becomes most ambitious. It imagines planets, celestial systems, terrain, domains, histories, exploration, settlement, robotics, automated infrastructure, and systems that can grow from small deterministic seeds into large simulated worlds.

This ambition is useful, but it is not current implementation scope. The right way to preserve it is not to build everything now. It is to prevent early architecture from making it impossible later.

The safest path is to define a small deterministic slice that exercises the same principles: a world state that can be advanced by process, inspected by tools, projected by presentation, serialized, replayed, validated, and extended by packs without losing authority.

The archive's worldgen ambition should therefore be used as a stress test. If an early architecture cannot support seeded generation, sparse persistence, replayable change, and future refinement, it may be too narrow. If it tries to implement the whole universe at once, it is too broad.

12. Civilization, Economy, Logistics, Institutions, and Signals

The archive often describes civilization-scale systems: economy, logistics, institutions, signals, robotics, infrastructure, factories, and social or operational structure. These ideas give Dominium its sense of depth.

They also create risk. If these systems are treated as vague lore, they will not help architecture. If they are treated as immediate requirements, they will overwhelm the current repo. The correct position is to treat them as long-horizon design intent that should inform boundary choices and future vertical slices.

The strongest practical reading is that Dominium should avoid designing only for direct player action. It should leave room for automated infrastructure, governed processes, signal flow, and systems that can be inspected and influenced through tools as well as through game surfaces.

These systems are valuable because they point toward indirect play and systemic consequences. A player might act through robots, infrastructure, tools, logistics, or institutional levers rather than direct character action alone. That possibility should inform the first slice, even if the first slice only proves a tiny part of it.

13. Documentation, Archive, and Corpus Governance

The archive has become part of the project. That is both valuable and hazardous. It preserves decisions, rationale, rejected options, prompt sequences, reports, and long-range ambition. It also contains duplicates, stale claims, generated summaries, and material that can look more authoritative than it is.

The project vision corpus solves part of this problem by separating source selection, source blocks, themes, synthesis, review queues, and validation reports. The external scratchpad corpus adds a second lens: it found a different package shape and gave a stronger product-ecosystem narrative.

The lesson is not to worship either corpus. The lesson is to use both as evidence while preserving current authority. A future book or spec should be easier to read than the archive, but stricter about uncertainty than a chat summary.

The correct archive posture is preservation without obedience. Old material should remain reachable because it contains rationale. It should also remain labelled because it contains stale assumptions. The book should make old knowledge useful without letting it bypass review.

This is also why future outputs should keep their jobs separate. A corpus preserves and sorts evidence. A book explains it. A spec turns accepted claims into constraints. A queue task authorizes work. Confusing those roles is one of the easiest ways for archive enthusiasm to become silent drift.

14. What Is Already Decided

Several decisions are settled enough to guide planning. Domino and Dominium should remain layered. Determinism, replay, provenance, authority ordering, explicit refusal, and process-only mutation are central. Presentation should project state, not own it. Generated outputs and archive material are advisory unless promoted by stronger sources.

Providers and external libraries are welcome, but they belong behind contracts. Workbench and AIDE should help governance rather than bypass it. Content and packs should extend behavior through registries and compatibility law rather than through hidden assumptions.

These are not final implementation designs. They are planning constraints. They tell future tasks what must not be violated while the exact APIs, products, and slices are still being decided.

These decisions should be treated as guardrails for future prompts. A future task that proposes renderer work, Workbench work, provider work, or gameplay work should show how it preserves deterministic truth, authority order, explicit refusal, and dependency direction before it changes anything.

15. What Is Not Yet Decided

The unresolved list is still substantial. The exact first playable slice is not settled. The final Domino/Dominium API boundary is not fully specified. Renderer and native GUI strategy remain future work. Workbench scope, provider ABI, package runtime, release identity, gameplay shape, and world simulation scope all need review.

The user still needs to choose which ambition becomes the first concrete vertical slice. A small deterministic simulation? A headless replay/provenance demo? A Workbench inspection surface? A pack/provider compatibility proof? A worldgen seed slice? Each option teaches something different.

The book should therefore prepare decisions, not pretend they are already made. It should turn the archive into a clear choice set.

The open decisions are not merely missing details. They determine the project's early shape. If the first slice is Workbench-first, the project learns governance and inspection early. If it is simulation-first, it learns state and replay early. If it is presentation-first, it risks looking usable before it proves authority.

16. Contradictions, Drift, and Caution Zones

The major contradiction is not that the archive disagrees on the core idea. It mostly agrees. The contradiction is between ambition and authorization. Many older conversations speak as if renderer, Workbench, product shell, provider runtime, gameplay, or world simulation work is ready to implement. Current queue state says otherwise.

A second drift zone is generated authority. Several prior books and reports became polished enough to feel official while still being derived. The project needs that polish for readability, but it must keep authority labels visible.

A third drift zone is provider enthusiasm. External engines and libraries can help, but if they become the place where law is defined, the project gives up the determinism and portability that made them useful in the first place.

The comparison corpus helps here because it makes one drift visible: broader archive packages can produce broader confidence. More files and more blocks do not automatically mean more truth. They mean more evidence to sort. Current authority still decides what can be relied on now.

17. The Practical Roadmap

The next practical move is review, not implementation. First, read the vision synthesis, current reality, design principles, decision docket, and contradictions reports. Second, choose which open decisions require user resolution. Third, verify external-only claims before using them in stricter planning.

After review, the safest repo work is narrow documentation refinement: clarify authority boundaries, decide which archive claims are useful, mark stale claims, and produce small future tasks with allowed paths and validation. Implementation should wait until a task explicitly opens it.

The first technical sequence should prove the spine before the breadth: deterministic state, process mutation, serialization, replay/provenance, provider refusal, and an inspectable surface. That spine can support later game, UI, renderer, and world ambitions.

A practical roadmap should also keep book, spec, and implementation separate. The book explains. A spec constrains. Implementation changes behavior. Moving directly from book to implementation would repeat the old problem of treating synthesis as authority.

The immediate roadmap is therefore deliberately unglamorous. It asks for reading, triage, verification, and decision work before visible product work. That is the right order for a project whose failures would come from hidden authority drift. Once the spine is proven, the more exciting surfaces have somewhere stable to attach.

18. The First Playable Slice Problem

The first playable slice is a decision problem because it must satisfy two competing needs. It must be small enough to build and validate. It must also be representative enough to protect the long-horizon vision.

A weak slice would prove only presentation. A stronger slice would prove lawful state change, deterministic replay, capability refusal, save compatibility, and a minimal projection surface. It could still be visually simple. The point is to exercise the project's unique constraints early.

The best candidate is probably not a full game loop yet. It is a deterministic, inspectable simulation slice with a narrow product surface: enough world state to matter, enough process to mutate lawfully, enough projection to see it, enough validation to trust it, and enough refusal to show where unsupported capabilities stop.

The slice should be chosen for proof value. A beautiful demo that does not prove replay, refusal, serialization, or authority will not answer the project's hardest questions. A modest demo that proves those things may be the better beginning, even if it is visually plain.

19. What Future Codex Tasks Should Do

Future Codex tasks should be smaller than the archive vision. They should name their authority sources, allowed paths, protected paths, outputs, validation, and non-goals. They should not turn advisory conversation material into live doctrine by accident.

The next tasks should probably be: accept or revise this book, triage user decisions, create a spec-book outline, verify external-only claims, define the first playable slice, and only then open narrow implementation or documentation-promotion work.

A good task after this book would ask for a spec-readiness pass: which book claims are narrative only, which can become requirements, which need verification, which require user decisions, and which are blocked by current queue state.

Future tasks should also avoid turning this book into another omnibus. The useful next step is not more bulk publication. It is a stricter map from narrative claims to spec candidates, from spec candidates to decisions, and from decisions to scoped validation tasks.

20. Closing Vision

Dominium's vision is compelling because it is not just bigger. It is more disciplined. It wants a world that can grow without losing proof, a product suite that can become useful without owning truth, and a tooling layer that can accelerate work without bypassing authority.

The project can use engines, renderers, libraries, providers, packs, native UI, Workbench, AIDE, and future automation. But those surfaces must remain arranged around the same center: deterministic, lawful, inspectable truth.

If that center holds, the long-horizon ambition can stay alive while the current repo moves in safe, reviewable steps. That is the real project vision: not a single artifact, but a disciplined path from substrate to product to world.

That discipline is what makes the vision more than a pile of ambitions. Dominium can pursue a large world, rich products, and flexible tooling because Domino keeps the center small enough to reason about. The project wins when breadth grows around a stable core rather than replacing it.

The archive shows many possible futures, but the best one is not the biggest unfiltered combination of them all. The best future is the one that keeps the recurring principles intact while choosing small proof steps. If the next phase preserves law, refusal, replay, provenance, and clear authority, then the project can afford to become larger over time.